

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук  
Образовательная программа «Прикладная математика и информатика»

УДК 004.75

**Отчет об исследовательском проекте на тему:**  
**Системы балансировки нагрузки в гибридных распределенных системах**

**Выполнил студент:**

группы #БПМИ236, 3 курса

Разухин Александр Сергеевич

**Принял руководитель проекта:**

Курочкин Илья Ильич

к.т.н., зав.лаб. Ц-1

ИППИ РАН

Москва 2026

# Содержание

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Аннотация</b>                                           | <b>3</b>  |
| <b>1 Введение</b>                                          | <b>4</b>  |
| 1.1 Описание предметной области . . . . .                  | 4         |
| 1.2 Релевантность и значимость . . . . .                   | 4         |
| 1.3 Цели и задачи . . . . .                                | 5         |
| 1.4 Предполагаемые результаты проекта . . . . .            | 5         |
| <b>2 Литературный обзор</b>                                | <b>6</b>  |
| 2.1 Обзор предметной области . . . . .                     | 6         |
| 2.2 Обзор существующих методов . . . . .                   | 7         |
| 2.3 Особенности BOINC . . . . .                            | 8         |
| 2.4 Сравнение подходов . . . . .                           | 8         |
| 2.5 Выводы из обзора . . . . .                             | 9         |
| <b>3 Моделирование системы</b>                             | <b>10</b> |
| 3.1 Выбор способа моделирования . . . . .                  | 10        |
| 3.2 Топология системы . . . . .                            | 10        |
| <b>4 Подходы к тестированию системы и метрики качества</b> | <b>11</b> |
| 4.1 Эвристики распределения задач . . . . .                | 11        |
| 4.2 Метрики оценки производительности . . . . .            | 11        |
| 4.3 Процесс тестирования . . . . .                         | 12        |
| <b>5 Разработка алгоритма балансировки</b>                 | <b>13</b> |
| 5.1 Планировщик BOINC по умолчанию . . . . .               | 13        |
| 5.2 Кастомный балансировщик задач . . . . .                | 13        |
| <b>6 Результаты тестирования и сравнение</b>               | <b>14</b> |
| 6.1 Стабильная среда без отказов . . . . .                 | 14        |
| 6.2 Нестабильная среда (1) . . . . .                       | 15        |
| 6.3 Нестабильная среда (2) . . . . .                       | 17        |
| <b>7 Заключение</b>                                        | <b>19</b> |



## **Аннотация**

При проектировании высокопроизводительных и распределенных систем одной из важнейших задач является выбор и настройка системы выдачи заданий (балансировщик нагрузки). При использовании гомогенных вычислительных узлов в кластере можно использовать готовые решения (slurm, torque и др.), но в гетерогенных системах готовые решения не будут работать из коробки. В распределенных системах из-за большой гетерогенности и других ограничений может возникнуть потребность разработки оригинального планировщика заданий. В распределенной вычислительной системе в качестве части ресурсов могут быть использованы облачные ресурсы или ресурсы кластеров. Уже были разработаны решения (к примеру, CluBoRun в грид-системе на платформе BOINC) по использованию ресурсов кластера в фоновом режиме. Предлагается разработать /модифицировать систему выдачи заданий для грид-системы на платформе BOINC, в которой присутствуют сегменты кластеров.

## **Ключевые слова**

Распределенные системы, балансировка нагрузки

# 1 Введение

## 1.1 Описание предметной области

Предметная область проекта охватывает распределенные системы с гетерогенной инфраструктурой с основными компонентами:

- BOINC (Berkeley Open Infrastructure for Network Computing) — платформа для добровольных вычислений, объединяющая ресурсы разнородных устройств (серверов, кластеров, персональных компьютеров) в единую сеть
- Кластеры — узлы, интегрированные в распределенную систему с целью повышения доступности
- Гетерогенные ресурсы — устройства с различными характеристиками: вычислительной мощностью, объемом памяти, доступностью и другими
- Балансировщик нагрузки — компонент, отвечающий за распределение вычислительных задач между узлами с учетом их возможностей и текущих состояний

Также предметная область имеет следующие особенности:

- Отсутствие централизованного контроля за узлами системы
- Разнородность ресурсов каждого из узлов
- Различные режимы работы узлов (не все устройства доступны на протяжении 100% времени)
- Различный объем задач, время выполнения которых может варьироваться от нескольких минут до нескольких дней
- Необходимость минимизации накладных расходов для балансировщика

## 1.2 Релевантность и значимость

Актуальность данного проекта обусловлена множеством существующих на данный момент проблем, таких как:

- 1 Современные вычислительные задачи такие, как анализ больших данных, биоинформатика, требуют огромного количества ресурсов, что способствует росту числа гетерогенных распределенных систем

- 2 Стандартные планировщики задач оптимизированы под гомогенные системы и не учитывают различия в производительности узлов и их доступность, что в гетерогенных системах приводит к простаиванию некоторых узлов или, наоборот, излишней загруженности отдельных
- 3 Необходимость развития динамических алгоритмов балансировки в противовес существующим решениям с их статическими правилами
- 4 Оптимизация всех вычислений, выполняющихся на платформе BOINC

### **1.3 Цели и задачи**

Целью данного проекта является разработка системы выдачи заданий для распределенной системы на платформе BOINC. Для ее достижения необходимо было выполнить следующие задачи:

- 1 Обзор систем балансировки нагрузки для кластеров и распределенных систем
- 2 Выбор системы моделирования и адаптация модели под специфику гибридной распределенной системы
- 3 Реализация/модификация системы балансировки нагрузки гибридной распределенной системы
- 4 Тестирование системы балансировки на модели гибридной распределенной системы
- 5 Анализ полученных результатов

### **1.4 Предполагаемые результаты проекта**

- 1 Подробное описание методики моделирования гибридной распределенной системы
- 2 Подробное описание системы балансировки нагрузки гибридной распределенной системы
- 3 Анализ результатов моделирования системы
- 4 Подготовка подробного отчета о проделанной работе и проведенном тестировании

## 2 Литературный обзор

### 2.1 Обзор предметной области

В своем начале Интернет имел очень мало потребителей, так что любой сервер мог выдерживать нагрузку даже самого популярного сайта, к тому же его падение не стало бы большой проблемой. Со временем люди были вынуждены заняться вопросами масштабирования ввиду растущей нагрузки. И если сначала проблема решалась увеличением числа процессоров, их улучшением, увеличением объема оперативной памяти, то после стало понятно, что есть потолок такого вертикального масштабирования, так что появилось понятие балансировки нагрузки. [2]

В общем случае, порядок работы балансировщика такой:

- 1 Получение входящих запросов от различных клиентов
- 2 Расчет размера входящих запросов и формирование очереди запросов
- 3 Периодическая проверка состояния серверов в пуле с помощью системы мониторинга
- 4 Использование стратегии / алгоритма / эвристики для выбора подходящего сервера под каждый входящий запрос

Сама идея балансировки нагрузки вносит в систему большое количество преимуществ:

- 1 Упрощенное отслеживания трафика
- 2 Улучшенное использование ресурсов
- 3 Балансировка нагрузки на сеть на основании загруженности серверов
- 4 Улучшенная доступность ресурсов
- 5 Уменьшение избыточного потребления ресурсов

[4]

Решения проблемы балансировки нагрузки делятся на 2 основных подхода в зависимости от того, принимает ли алгоритм решения на основе текущего состояния системы или нет: статический и динамический.

Статический подход предполагает наличие знаний о статусе системы, требованиях к ресурсам, времени связи и другим показателям. Балансировка при таком подходе достигается путем создания связей между списком процессоров и списком задач таким образом, чтобы достигалась максимальная производительность. Хотя назначения могут быть как детерминированными, так

и вероятностными, текущее состояние системы не учитывается ни в одном из вариантов организации балансировки. Это существенный недостаток статических подходов, который оказывает существенное влияние на общую производительность из-за непредсказуемости колебаний нагрузки на систему. Некоторые методы, используемые при статической балансировке: перечисление и поиск в пространстве решений, теория графов, теория очередей.

При динамическом подходе, решения о балансировке нагрузки принимаются на основе текущего состояния системы; задачам разрешается динамически перемещаться с перегруженного узла на недостаточно загруженный узел для более быстрой обработки. Способность реагировать на изменения в системе является основным преимуществом динамического подхода к балансировке нагрузки. Хотя найти динамическое решение намного сложнее, чем статическое, динамическая балансировка нагрузки может повысить производительность за счет своей адаптивности. Далее сосредоточимся именно на таких подходах.

Механизм управления динамической балансировкой может быть распределенным, когда ответственность за балансировку распределяется между узлами, и нераспределенным, когда ответственность за балансировку лежит на одном или нескольких узлах; включает централизованный (один центральный узел управляет балансировкой всей системы) и полу распределенный (узлы делятся на кластеры с центральными узлами для балансировки внутри кластера) подходы. [1]

## 2.2 Обзор существующих методов

Рассмотрим методы балансировки нагрузки, применимые к распределённым вычислительным системам с акцентом на гибридные конфигурации (платформы типа BOINC + стабильные кластерные сегменты). Среди критериев для них можно выделить: простота мониторинга, устойчивость к churn (неконтролируемое подключение / отключение узлов), гетерогенность ресурсов, величина накладных расходов, масштабируемость. Всего можно выделить несколько групп подходов к балансировке нагрузки:

- Классические статические методы (Round Robin, Random Strategy) — просты в мониторинге, однако не учитывают размер и время выполнения задач
- Классические динамические методы (Least-Connection) — также просты в мониторинге, но не учитывает общую нагрузку на узлы
- Адаптивные методы — балансировка происходит на основе метрик, вычисляемых в реальном времени; требуют постоянных вычислений

- Предиктивные ML подходы — планировщики, которые хорошо работают даже в сложных условиях, но требуют времени и данных для обучения модели [5]
- Существующие решения для BOINC (CluBoRun) — механизмы верификации, учета доступности, отслеживания churn и другие [3]

Большинство классических и оптимизационных алгоритмов работают при допущении, что узлы доступны и их характеристики относительно стабильны. В средах типа BOINC это допущение неприменимо: изменчивость системы и разнородность устройств приводит к частому перераспределению задач между узлами. Если в гомогенных системах перенос задач — относительно стандартный процесс, в гибридных системах такой подход может привести к серьёзному росту накладных расходов из-за неоднородности устройств, поэтому в подобных системах правильным подходом считается исключение или сведение к минимум подобныз операций. Также нужно отметить, что для гибридных систем важно сохранять баланс между количеством собираемых метрик и их полезности для планировщика, что труднодостижимо при подходах с глобальным мониторингом узлов. [4]

## 2.3 Особенности BOINC

Платформа BOINC использует модель *client-initiated*: вычислительный узел сам запрашивает работу через RPC. Scheduler сканирует shared-memory массив feeder, оценивает совместимость workunit с host и ранжирует кандидатов по score. Назначенная задача не мигрирует на другой host до timeout или повторной репликации. Поле `rsc_frops_est` задаёт оценку сложности, которая может расходиться с реальным временем выполнения — это важный источник ошибок для любой adaptive-политики. Решение CluBoRun [3] интегрирует кластерные ресурсы в BOINC-проект, но не решает глобальную балансировку по классам устройств (кластер / desktop / mobile).

## 2.4 Сравнение подходов

Поймем, насколько применимы указанные подходы к гибридным системам и выделим их недочеты:

Таблица 2.1: Сравнение групп подходов к балансировке нагрузки в гибридных системах

| Группа методов            | Тип                             | Требования мониторинга | Цель подхода                                   | Минусы                            |
|---------------------------|---------------------------------|------------------------|------------------------------------------------|-----------------------------------|
| Классические статические  | Простые                         | Низкие                 | Простое планирование                           | Не учитывают размер задач         |
| Классические динамические | Реактивные                      | Низкие/средние         | Сократить число одновременных задач на узле    | Не учитывает CPU-time узла        |
| Адаптивные методы         | Feedback-driven                 | Средние/высокие        | Низкая задержка и оптимальная нагрузка на узлы | Высокие накладные расходы         |
| Предиктивные подходы      | ML Обучаемые                    | Высокие                | Минимизация целевой функции ошибки             | Требуют данных и обучения         |
| CluBORun                  | Интеграция кластера / устройств | Низкие/средние         | Использование idle ресурсов, учёт доступности  | Не решает глобальную балансировку |

## 2.5 Выводы из обзора

- На данный момент существует широкий набор методов балансировки, однако немногие работы систематически исследуют их поведение в гибридных системах
- Основные пробелы: недостаточная оценка влияния churn, упрощённое моделирование сетевых затрат при распределении задач, отсутствие оценки накладных расходов мониторинга и планировщика

Данная работа направлена на устранение выявленных ограничений за счёт оптимизации накладных расходов планировщика, более точного учёта динамической доступности узлов и повышения общей производительности системы при рациональном использовании вычислительных ресурсов.

## 3 Моделирование системы

### 3.1 Выбор способа моделирования

### 3.2 Топология системы

Тестовый стенд состоит из 6 хостов-устройств с различными характеристиками:

- 2 кластерных элемента (4 CPU, 4 Gb RAM)
- 1 устройство ПК (2 CPU, 2 Gb RAM)
- 1 маломощный узел (1 CPU, 1 Gb RAM)
- 2 элемента - мобильных устройств (0.5 CPU 0.5 Gb RAM)

Каждое устройство представляет собой независимый вычислительный хост; с точки зрения технической реализации каждый хост - это отдельный **Docker** контейнер, который конфигурационно настроен под конкретное число CPU. Данный выбор хостов с их настройками был обусловлен следующими требованиями к тестовому стенду:

- Стенд обязан содержать разнородные с точки зрения мощности устройства для моделирования гибридной среды
- Стенд обязан содержать как высокопроизводительные элементы (кластеры), так и низкопроизводительные (мобильные устройства)
- Стенд обязан иметь несколько одинаковых устройств для проверки балансировки нагрузки между однородными хостами (распределение задач между одинаковыми с точки зрения мощности клиентами)

Таким образом и была сформирована модель системы, используемая при тестировании

## 4 Подходы к тестированию системы и метрики качества

### 4.1 Эвристики распределения задач

В литературе помимо классических подходов рассматриваются некоторые эвристики, которые были смоделированы в данном проекте для сравнения с разработанным методом:

- LPT (Longest Processing Time first) - длинные задачи назначаются первыми; при нескольких параллельно работающих однотипных хостах снижает общее время работы, но может ухудшить качество распределения и среднее время отклика для коротких задач
- SJF (Shortest Job First) - короткие задачи получают приоритет; минимизирует среднее время отклика, но крупные задачи могут долго простаивать в очереди

Кластерные планировщики (Slurm, Torque) оптимизированы под гомогенные очереди с централизованным контролем и статическим планированием; они не учитывают pull-модель BOINC и отказы узлов, поэтому не применимы к гибридной системе, по этой причине они не попали в предметную область данного проекта. Таким образом, кроме алгоритма балансировки BOINC по умолчанию были добавлены в тестирование упомянутые выше LPT, SJF, а также классические подходы: Round Robin, Random и Weighted Least Loaded.

### 4.2 Метрики оценки производительности

Выбор метрик обусловлен необходимостью учета как общих показателей производительности, так и особенностей гетерогенной вычислительной среды.

- Пропускная способность (Throughput) – число выполненных задач за единицу времени. Формула:  $T = \frac{N_{\text{completed}}}{\Delta t}$ .
- Время выполнения задачи – среднее, медиана, перцентили (95%, 99%). Для каждой задачи  $i$ :  $R_i = t_{\text{finish}}^{(i)} - t_{\text{arrival}}^{(i)}$ . Среднее:  $\bar{R} = \frac{1}{N} \sum_i R_i$ .
- Загруженность узлов – для узла  $j$ :  $u_j = \frac{\text{busy\_cpu\_time}_j}{\text{observation\_time}}$ . Сводные метрики: среднее по узлам  $\bar{u}$  и коэффициент вариации  $CoV = \frac{\sigma(u_j)}{\bar{u}}$  (показывает балансировку).
- Общее время завершения набора задач –  $C = \max_i \{t_{\text{finish}}\} - \min_i \{t_{\text{arrival}}\}$

- Накладные расходы планировщика – среднее время, затраченное на принятие решения по задаче, и доля от общего времени:

$$OH_{avg} = \frac{1}{N} \sum_i t_{sched}^{(i)}, \quad OH_{\%} = \frac{\sum_i t_{sched}^{(i)}}{T_{wall}} \cdot 100\%.$$

Такой набор метрик позволяет оценить как производительность системы, так и качество балансировки нагрузки в условиях гибридной среды.

### 4.3 Процесс тестирования

Система тестировалась в разных условиях для приближения к реальной жизни. Стенд позволяет настроить отказы некоторых хостов (моделирование churn), их длительность и частоту. Churn реализован через временную остановку контейнера, что допустимо при подобном локальном моделировании системы. В рамках теста были смоделированы стабильная среда и среда с периодическими кратковременными отказами хостов. При каждом запуске автоматически вычислялись метрики качества для дальнейшего анализа, а также сохранялись текущие конфигурации системы для воспроизводимости результатов. Отметим про воспроизводимость, что в политике Random использовались псевдослучайные числа для детерменированного поведения системы от запуска к запуску (при равенстве основных параметров). Каждый раз прогонялись все политики, после чего метрики агрегировались и анализировались в рамках оптимизации целевого алгоритма балансировки.

В качестве источника задач использовалось встроенное приложение BOINC *uppercase*, которое генерировало задачи разной сложности через параметр *cpu\_time*. Всего было 3 типа задач: *short* (*cpu\_seconds* = 5), *medium* (*cpu\_seconds* = 30), *long* (*cpu\_seconds* = 120). Идея в том, чтобы не только оптимизировать распределение задач между узлами, чтобы все исполняли примерно одно количество задач, но и повысить честность такого распределения, а именно исполнение более тяжелых задач мощными узлами и наоборот - более легкие задачи оставлять маломощным узлам. Задачи *medium* были добавлены для разнообразия выборки.

## **5 Разработка алгоритма балансировки**

### **5.1 Планировщик BOINC по умолчанию**

### **5.2 Кастомный балансировщик задач**

Основной идеей было взять за основу базовый планировщик и оптимизировать его под гибридную систему для лучших метрик.

## 6 Результаты тестирования и сравнение

Суммарно было X прогонов для разных сред и настроек отказов. В таблицах обозначены отражены следующие свойства и метрики:

- Policy - текущая политика
- Throughput - пропускная способность (задач в секунду)
- Response - среднее время ожидания взятия задачи в работу (в секундах)
- Reponse p95 - 95-ый перцентиль прошлой метрики
- Makespan - общее время выполнения набора задач (в секундах)
- Load CV - ковариация нагрузки на узлы
- Jain - равномерность распределения задач

Ниже эксперименты и короткие выводы по ним:

### 6.1 Стабильная среда без отказов

Таблица 6.1: Метрики для стабильной среды

| Policy                | Throughput | Response | p95   | Makespan | Load CV | Jain  |
|-----------------------|------------|----------|-------|----------|---------|-------|
| baseline              | 0.117      | 124.2    | 187.5 | 252      | 0.368   | 0.725 |
| round_robin           | 0.116      | 117.7    | 197.5 | 266      | 0.449   | 0.812 |
| weighted_least_loaded | 0.109      | 117.6    | 206.5 | 274      | 0.711   | 0.761 |
| sjf                   | 0.109      | 119.2    | 214.0 | 272      | 0.453   | 0.818 |
| random                | 0.111      | 156.9    | 220.0 | 269      | 0.350   | 0.628 |
| lpt                   | 0.095      | 144.8    | 190.3 | 324      | 0.451   | 0.688 |
| hybrid                | 0.096      | 140.4    | 217.8 | 338      | 0.758   | 0.923 |

Как видим, кастомный метод проиграл планировщику по умолчанию (да и остальным политикам) по всем временным метрикам: продолжительность работы (338 секунд, на 30% больше, чем у baseline), пропускная способность (в среднем на 10-15% хуже остальных), время простоя задач (в p95 хуже него только random). Классические методы Round Robin и Weighted Least Loaded приближены по качеству к baseline, однако тот показывает лучшую ковариацию нагрузки (0.368).

Эвристики также заметно проигрывают по скорости работы планировщику ”из коробки”. Единственная метрика, по которой самописный планировщик опережает другие политики - это равномерность распределения задач по хостам - 0.923, что на 15-30% лучше остальных алгоритмов балансировки. Ниже представлена статистика по хостам:

Таблица 6.2: Число завершённых задач по хостам

| Policy                | cluster-01 | cluster-02 | desktop-01 | low-power-01 | phone-01 | phone-02 | $\Sigma$ |
|-----------------------|------------|------------|------------|--------------|----------|----------|----------|
| baseline              | 12         | 10         | 5          | 3            | 4        | 2        | 36       |
| round_robin           | 10         | 10         | 5          | 4            | 3        | 4        | 36       |
| weighted_least_loaded | 9          | 12         | 5          | 4            | 3        | 3        | 36       |
| sjf                   | 9          | 10         | 7          | 3            | 3        | 4        | 36       |
| random                | 14         | 10         | 6          | 2            | 2        | 2        | 36       |
| lpt                   | 11         | 12         | 5          | 2            | 4        | 2        | 36       |
| hybrid                | 8          | 8          | 6          | 3            | 6        | 5        | 36       |

По данной таблице становится очевидным преимущество написанного алгоритма - за счет своей архитектуры он не позволяет кластерам забирать большую часть задач для исполнения, а равномерно распределяет их по всем хостам (суммарно на оба кластера приходится 44% задач против 61% у планировщика по умолчанию). Качество такого распределения еще подробнее можно увидеть на таблице ниже со статистикой разбора задач каждым хостом по типам:

Таблица 6.3: Число завершённых задач по типам и хостам

|                 | Host         |       |        |      |          |               | Host         |       |        |      |          |
|-----------------|--------------|-------|--------|------|----------|---------------|--------------|-------|--------|------|----------|
|                 |              | short | medium | long | $\Sigma$ |               |              | short | medium | long | $\Sigma$ |
| <b>baseline</b> | cluster-01   | 7     | 4      | 1    | 12       | <b>hybrid</b> | cluster-01   | 3     | 4      | 1    | 8        |
|                 | cluster-02   | 5     | 2      | 3    | 10       |               | cluster-02   | 1     | 2      | 5    | 8        |
|                 | desktop-01   | 2     | 2      | 1    | 5        |               | desktop-01   | 2     | 4      | 0    | 6        |
|                 | low-power-01 | 1     | 2      | 0    | 3        |               | low-power-01 | 1     | 2      | 0    | 3        |
|                 | phone-01     | 3     | 1      | 0    | 4        |               | phone-01     | 6     | 0      | 0    | 6        |
|                 | phone-02     | 0     | 1      | 1    | 2        |               | phone-02     | 5     | 0      | 0    | 5        |
|                 | $\Sigma$     | 18    | 12     | 6    | 36       |               | $\Sigma$     | 18    | 12     | 6    | 36       |

Здесь преимущества становятся более очевидными: планировщик по умолчанию отдает почти все short-задачи (66%) мощным кластерам, так что слабые устройства вынуждены разбирать средние и тяжелые задачи, тогда как в случае с кастомным алгоритмом кластеры забирают все long-задачи и половину medium, оставляя легкие маломощным узлам.

## 6.2 Нестабильная среда (1)

При данном запуске были смоделированы отказы каждые CHURN\_ONLINE\_SECONDS=25 секунд на CHURN\_OFFLINE\_SECONDS=12 секунд для тестирования приближенной к реально-

сти модели, когда в каждый момент времени доступны не все узлы. Рассмотрим аналогичные построенным ранее таблицы для данного прогона:

Таблица 6.4: Метрики для нестабильной среды - 1

| Policy                | Throughput | Response | p95   | Makespan | Load CV | Jain  |
|-----------------------|------------|----------|-------|----------|---------|-------|
| baseline              | 0.121      | 119.4    | 198.8 | 243      | 0.329   | 0.806 |
| round_robin           | 0.112      | 120.2    | 209.3 | 277      | 0.465   | 0.857 |
| weighted_least_loaded | 0.103      | 133.9    | 229.0 | 295      | 0.459   | 0.893 |
| sjf                   | 0.110      | 129.4    | 223.5 | 290      | 0.587   | 0.851 |
| random                | 0.085      | 161.5    | 213.3 | 363      | 0.399   | 0.610 |
| lpt                   | 0.122      | 141.4    | 196.8 | 237      | 0.344   | 0.617 |
| hybrid                | 0.090      | 146.8    | 229.5 | 344      | 0.654   | 0.871 |

Таблица 6.5: Число завершённых задач по хостам

| Policy                | cluster-01 | cluster-02 | desktop-01 | low-power-01 | phone-01 | phone-02 | $\Sigma$ |
|-----------------------|------------|------------|------------|--------------|----------|----------|----------|
| baseline              | 11         | 9          | 5          | 4            | 4        | 3        | 36       |
| round_robin           | 9          | 9          | 7          | 4            | 4        | 3        | 36       |
| weighted_least_loaded | 8          | 9          | 7          | 4            | 4        | 4        | 36       |
| sjf                   | 9          | 10         | 5          | 4            | 4        | 4        | 36       |
| lpt                   | 12         | 13         | 5          | 2            | 2        | 2        | 36       |
| hybrid                | 8          | 10         | 5          | 3            | 5        | 5        | 36       |
| random                | 9          | 15         | 6          | 2            | 2        | 2        | 36       |

Таблица 6.6: Число завершённых задач по типам и хостам

| baseline | Host         | short | medium | long | $\Sigma$ |
|----------|--------------|-------|--------|------|----------|
|          | cluster-01   | 7     | 4      | 1    | 11       |
|          | cluster-02   | 4     | 2      | 3    | 9        |
|          | desktop-01   | 2     | 2      | 1    | 5        |
|          | low-power-01 | 3     | 1      | 0    | 4        |
|          | phone-01     | 2     | 1      | 1    | 4        |
|          | phone-02     | 0     | 3      | 0    | 3        |
|          | $\Sigma$     | 18    | 12     | 6    | 36       |

| hybrid | Host         | short | medium | long | $\Sigma$ |
|--------|--------------|-------|--------|------|----------|
|        | cluster-01   | 3     | 0      | 5    | 8        |
|        | cluster-02   | 3     | 6      | 1    | 10       |
|        | desktop-01   | 1     | 4      | 0    | 5        |
|        | low-power-01 | 1     | 2      | 0    | 3        |
|        | phone-01     | 5     | 0      | 0    | 5        |
|        | phone-02     | 5     | 0      | 0    | 5        |
|        | $\Sigma$     | 18    | 12     | 6    | 36       |

Видно, что все преимущества самописного алгоритма в стабильной среде успешно перенесли на нестабильную с постоянными отказами: качество распределения задач (0.871) так же лучше чем у любой другой политики; кластеры так же забирают на себя все сложные задачи, оставляя легкие для более слабых узлов; он все еще проигрывает по времени (makespan 344 секунды, что значительно больше, чем у остальных), но потери в ключевых метриках составляют около 10-15%, что является по своей сути компромиссом для лучшего распределения задач.

## 6.3 Нестабильная среда (2)

При данном запуске были смоделированы отказы каждые CHURN\_ONLINE\_SECONDS=40 секунд на CHURN\_OFFLINE\_SECONDS=10 секунд, то есть недоступность в 20% против 33% в прошлом эксперименте. Статистика для данного запуска:

Таблица 6.7: Метрики для нестабильной среды - 2

| Policy                | Throughput | Response | p95   | Makespan | Load CV | Jain  |
|-----------------------|------------|----------|-------|----------|---------|-------|
| baseline              | 0.119      | 134.9    | 186.3 | 255      | 0.477   | 0.844 |
| round_robin           | 0.102      | 127.9    | 217.5 | 285      | 0.591   | 0.850 |
| weighted_least_loaded | 0.111      | 127.0    | 221.0 | 276      | 0.519   | 0.857 |
| sjf                   | 0.103      | 128.8    | 220.5 | 294      | 0.608   | 0.857 |
| random                | 0.114      | 145.3    | 216.3 | 262      | 0.450   | 0.610 |
| lpt                   | 0.113      | 150.3    | 213.8 | 264      | 0.320   | 0.610 |
| hybrid                | 0.097      | 142.7    | 214.5 | 335      | 0.758   | 0.871 |

Таблица 6.8: Число завершённых задач по хостам

| Policy                | cluster-01 | cluster-02 | desktop-01 | low-power-01 | phone-01 | phone-02 | $\Sigma$ |
|-----------------------|------------|------------|------------|--------------|----------|----------|----------|
| baseline              | 10         | 9          | 5          | 3            | 5        | 4        | 36       |
| round_robin           | 9          | 10         | 5          | 4            | 4        | 4        | 36       |
| weighted_least_loaded | 9          | 9          | 7          | 4            | 4        | 3        | 36       |
| sjf                   | 9          | 9          | 7          | 4            | 3        | 4        | 36       |
| random                | 14         | 11         | 5          | 2            | 2        | 2        | 36       |
| lpt                   | 14         | 11         | 5          | 2            | 2        | 2        | 36       |
| hybrid                | 8          | 10         | 5          | 3            | 5        | 5        | 36       |

Таблица 6.9: Число завершённых задач по типам и хостам

| baseline | Host         | short | medium | long | $\Sigma$ |
|----------|--------------|-------|--------|------|----------|
|          | cluster-01   | 6     | 2      | 2    | 10       |
|          | cluster-02   | 4     | 2      | 3    | 9        |
|          | desktop-01   | 1     | 3      | 1    | 5        |
|          | low-power-01 | 0     | 3      | 0    | 3        |
|          | phone-01     | 4     | 1      | 0    | 5        |
|          | phone-02     | 3     | 1      | 0    | 4        |
|          | $\Sigma$     | 18    | 12     | 6    | 36       |

| hybrid | Host         | short | medium | long | $\Sigma$ |
|--------|--------------|-------|--------|------|----------|
|        | cluster-01   | 0     | 3      | 5    | 8        |
|        | cluster-02   | 5     | 4      | 1    | 10       |
|        | desktop-01   | 2     | 3      | 0    | 5        |
|        | low-power-01 | 1     | 2      | 0    | 3        |
|        | phone-01     | 5     | 0      | 0    | 5        |
|        | phone-02     | 5     | 0      | 0    | 5        |
|        | $\Sigma$     | 18    | 12     | 6    | 36       |

В этом эксперименте baseline продолжает лидировать по makespan (255 секунд) и пропускной способности (0.119), а кастомный алгоритм сохраняет лучшее качество распределения (0.871)

и снова проигрывает по makespan на 30%. По сравнению с прошлым экспериментом самописный алгоритм теряет меньше по пропускной способности (-19% против -26% ранее).

Отметим, что по таблице может сложиться мнение о некачественном распределении задач, так как долгие задачи между кластерами делятся в отношении 1:5, что не совсем соответствует поставленной задаче избавиться от подобных проблем. Однако если смотреть на картину в целом, то видно, что низкое число long-задач менее загруженный кластер компенсирует большим числом выполненных short- и medium-задач, что прослеживается во всех запусках.

В общем и целом по итогам тестирования можно сделать вывод, что алгоритм с основной своей задачей справляется, а именно честное распределение задач при

## **7 Заключение**

## Список литературы

- [1] Ali M. Alakeel. «A Guide to Dynamic Load Balancing in Distributed Computer Systems». B: *International Journal of Computer Science and Network Security* 1.2 (2010).
- [2] Tony Bourke. *Server Load Balancing*. O'Reilly & Associates, 2001.
- [3] Dolgov A. Kurochkin I. «Load balancing in grid systems from personal devices on the BOINC platform». B: *XIV All-russian multi-conference on management issues of the MCPU-2021*. 2021.
- [4] Jerry Gao Mazedur Rahman Samira Iqbal. «Load Balancer as a Service in Cloud Computing». B: *IEEE 8th International Symposium on Service Oriented System Engineering (SOSE)*. 2014.
- [5] Lun WANG. «Low-Latency, High-Throughput Load Balancing Algorithms». B: *Journal of Computer Technology and Applied Mathematics* 10.6 (2024), с. 153—160.